

LangGraph 底层原理：它是怎么把 LLM 变成一台状态机的

原创 Jameszyh James的成长日记 2026年4月24日 09:26 广东

大家好，我是James。

上一篇我们把 LangChain Memory 模块拆了个底朝天，搞清楚了截断、总结、检索三种记忆方案各自的适用场景。这一篇我们进入 LangGraph 的核心地带——它的底层执行引擎。

你有没有想过：为什么同样是调用 LLM，用了 LangGraph 之后，AI 就能「想一步、停一步、判断一步」？就能在工具调用和 LLM 推理之间来回切换，还不会乱套？

答案就藏在它的状态机设计里。

01 先说说 LangChain 的老问题

LangChain 的早期架构是线性的——Chain。



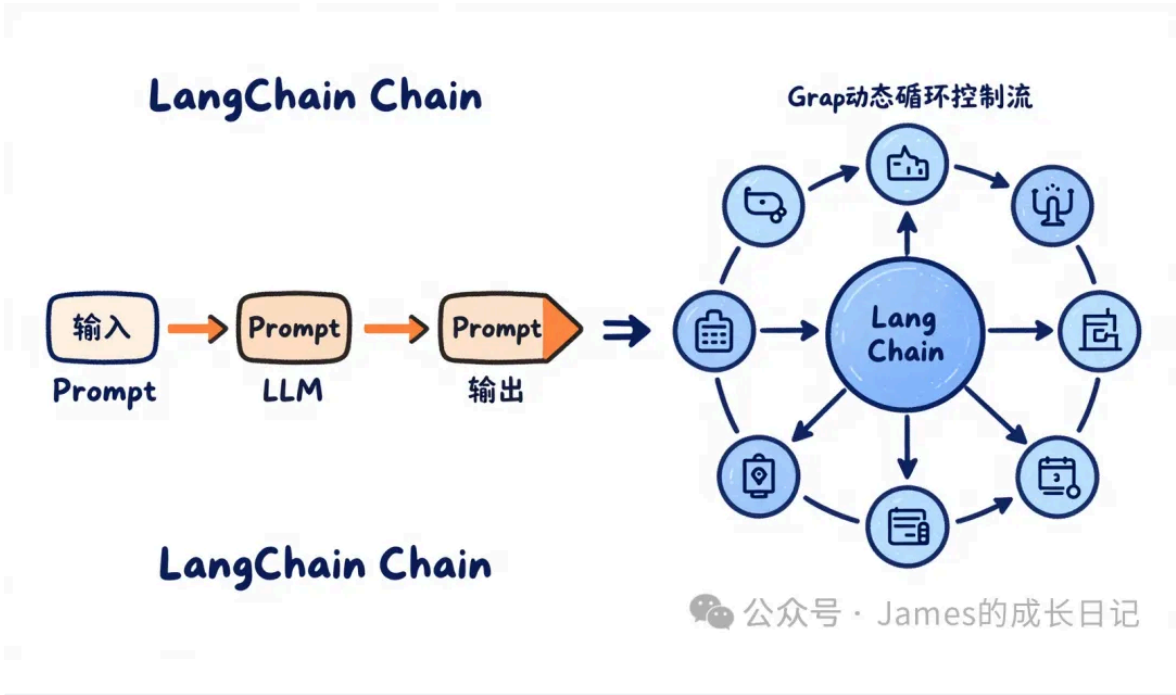
这个模型干净利落，但有个致命弱点：**控制流是固定的**。

一旦你想实现「如果 LLM 觉得需要查数据库就查，不需要就跳过」，或者「出错了重试三次」，你会发现原来的 Chain 根本放不下这些逻辑。要么把判断逻辑硬编码在节点里，要么用 Python 的 if/else 在外层手动管流程——乱成一锅粥。

现实中的 Agent 场景根本不是线性的：

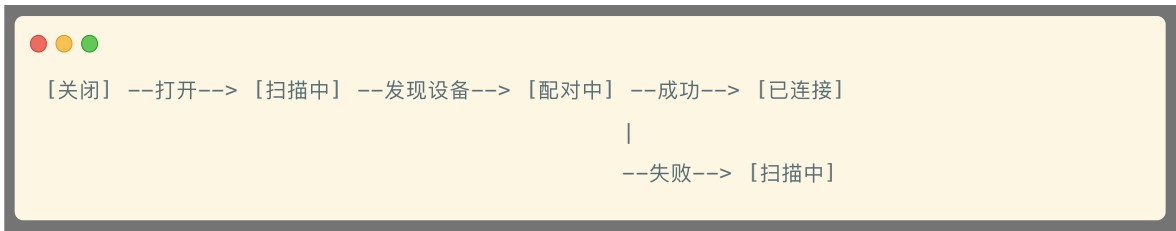
- 调工具 → 看结果 → 决定调下一个还是直接回答
- 生成草稿 → 自我审查 → 不行就重写
- 多轮对话 → 根据意图走不同分支

这些都需要**带状态的循环控制流**。这正是 LangGraph 解决的核心问题。



02 状态机是什么：用最简单的例子解释

状态机（State Machine）不是什么新概念。你手机的蓝牙就是个状态机：



三个核心要素：



LangGraph 就是把这套机制套在了 LLM 上：

- **State** = 对话历史 + 工具结果 + 中间变量
- **Node** = LLM 调用 / 工具执行 / 业务逻辑
- **Edge** = 普通跳转 / 条件分支（Conditional Edge）

03 StateGraph 执行引擎：它的核心长什么样

打开 LangGraph 的源码（Python 版 `langgraph/graph/state.py`，JS 版同理），你会看到 `StateGraph` 类维护了几个核心数据结构：

StateGraph 内部	
<code>nodes</code>	<code>Map<名称, 函数></code>
<code>edges</code>	<code>Map<from, to[]></code>
<code>conditional_</code>	<code>Map<from, (state)=>节点名></code>
<code>edges</code>	
<code>channels/</code>	每个状态字段的 Reducer
<code>schema</code>	

当你调用 `graph.compile()` 时，它做了什么？

<code>compile()</code> 阶段
├ 验证图结构（有没有孤立节点？有没有到 END 的路径？）
├ 构建邻接表（预计算每个节点的后继）
├ 初始化 Channels（每个状态字段注册 Reducer）
└ 返回 CompiledGraph（可以 <code>invoke/stream</code> ）

`compile()` 本质上是把你「描述的图」变成「可执行的调度器」。

04 一次完整执行：从 invoke 到节点运行的全流程

来看最简单的例子，一个单节点 LLM Agent：

```
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage } from "@langchain/core/messages";

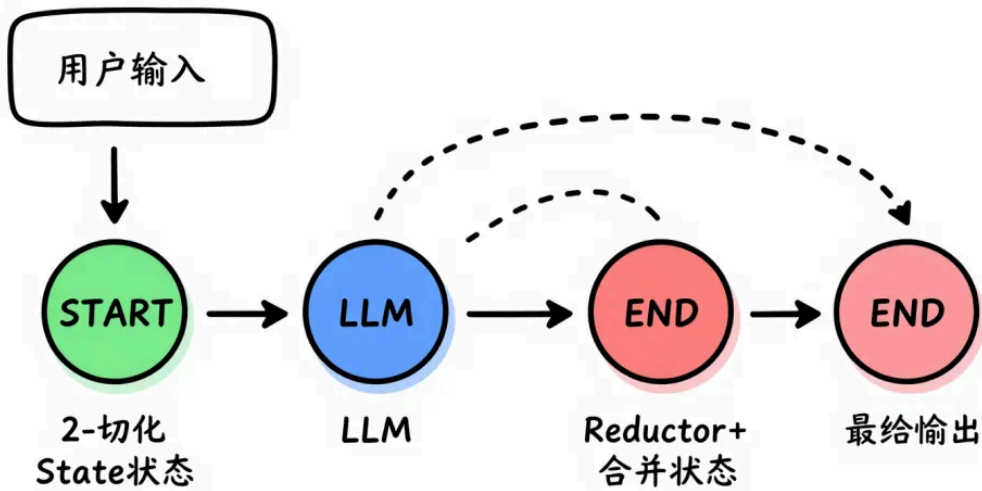
// 1. 定义状态结构
const AgentState = Annotation.Root({
  messages: Annotation<HumanMessage[]>({
    reducer: (prev, next) => [...prev, ...next],
    default: () => [],
  })
});
```

```
    }},  
  });  
  
  // 2. 初始化 LLM  
  const llm = new ChatOpenAI({ model: "gpt-4o-mini" });  
  
  // 3. 定义节点函数  
  async function callLLM(state: typeof AgentState.State) {  
    const response = await llm.invoke(state.messages);  
    return { messages: [response] };  
  }  
  
  // 4. 构建图  
  const graph = new StateGraph(AgentState)  
    .addNode("llm", callLLM)  
    .addEdge(START, "llm")  
    .addEdge("llm", END)  
    .compile();  
  
  // 5. 运行  
  const result = await graph.invoke({  
    messages: [new HumanMessage("你好，介绍一下自己")],  
  });
```

执行流程逐步拆解：



→ 执行结束
|
▼
6. 返回最终 State



公众号 · James的成长日记

05 Reducer：状态更新的核心机制

这是很多人没搞清楚的地方。

节点函数返回的不是「新 State」，而是「State 的更新片段」。LangGraph 用 **Reducer** 把旧状态和更新片段合并。

```
// 三种常见 Reducer 写法

// 方式1: 用内置 messagesStateReducer (追加消息)
const State1 = Annotation.Root({
  messages: Annotation<BaseMessage[]>({
    reducer: (prev, next) => [...prev, ...next],
  }),
});

// 方式2: 覆盖 (最新值覆盖旧值)
const State2 = Annotation.Root({
  step: Annotation<number>({
    reducer: (_, next) => next, // 直接替换
  }),
});

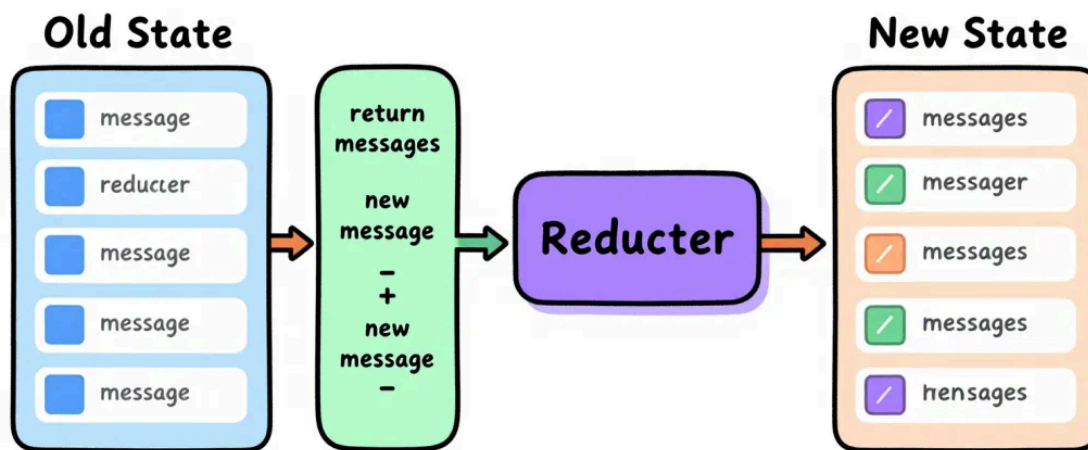
// 方式3: 累加计数
```

```
const State3 = Annotation.Root({
  callCount: Annotation<number>({
    reducer: (prev, next) => prev + next,
    default: () => 0,
  }),
});
```

Reducer 的执行时机：

```
节点返回 { key: value }
    |
    v
对每个 key, 找到对应 Reducer
newState[key] = reducer(oldState[key], value)
    |
    v
生成新 State 快照
(旧 State 不变, 不可变数据结构)
```

关键点：State 是不可变的。 每次节点执行都产生一个新的 State 快照，旧快照被保留（这是 Checkpoint 能实现的基础，后面讲）。



State不可变，每次产生新快照

公众号 · James的成长日记

06 图的调度器：它怎么决定下一步去哪

这是 LangGraph 最有意思的部分——调度器（Scheduler）。

调度器本质上是一个**事件循环**：



```
while 当前节点 != END:
    1. 执行当前节点 node(state) → partial_update
    2. 用 Reducer 合并状态 → new_state
    3. 查询当前节点的出边
        - 普通边：直接去下一个节点
        - 条件边：调用路由函数 router(new_state) → 返回节点名
    4. 把下一个节点加入执行队列
    5. 取队列头 → 重复
```

条件边（Conditional Edge）长这样：

```
import { StateGraph, START, END } from "@langchain/langgraph";

// 路由函数：根据 state 返回下一个节点名
function routeAfterLLM(state: typeof AgentState.State): string {
    const lastMessage = state.messages[state.messages.length - 1];

    // 如果 LLM 想调工具
    if("tool_calls" in lastMessage && lastMessage.tool_calls?.length) {
        return "tools"; // → 去工具节点
    }
    return END; // → 直接结束
}

const graph = new StateGraph(AgentState)
    .addNode("llm", callLLM)
    .addNode("tools", callTools)
    .addEdge(START, "llm")
    .addConditionalEdges("llm", routeAfterLLM, {
        tools: "tools", // 映射：路由函数返回值 → 节点名
        [END]: END,
    })
    .addEdge("tools", "llm") // 工具执行完回到 LLM
    .compile();
```

执行路径示意：

```
START → llm → [判断] → 需要工具?
                        Yes → tools → llm → [判断] → 不需要了?
                                                Yes → END
                        No  → END
```

这就是 ReAct Agent 的本质：一个带条件边的循环图。

07 并行执行：Fan-out / Fan-in 模式

LangGraph 支持一个节点连到多个后继节点，实现并行执行。

```
// Fan-out: 一个节点触发多个并行节点
const parallelGraph = new StateGraph(AgentState)
  .addNode("start", startNode)
  .addNode("search_web", searchWeb)    // 并行
  .addNode("search_db", searchDB)      // 并行
  .addNode("merge", mergeResults)      // 合并
  .addEdge(START, "start")
  .addEdge("start", "search_web")      // 同时出发
  .addEdge("start", "search_db")
  .addEdge("search_web", "merge")      // 都完成后汇聚
  .addEdge("search_db", "merge")
  .addEdge("merge", END)
  .compile();
```

调度器处理并行的方式：

```
start 节点执行完毕
  |
  ├──→ search_web (加入执行队列)
  └──→ search_db (加入执行队列)

执行队列: [search_web, search_db]
同时调度 (异步并发执行)

两者都完成后:
merge 节点的所有前驱都就绪 → merge 入队
  |
  ▼
merge 执行 → END
```

这个 Fan-out/Fan-in 模式，是并行搜索、多 Agent 协作的基础。

08 编译产物：CompiledGraph 里藏了什么

`compile()` 返回的 `CompiledGraph` 暴露了四个主要接口：




```
interface CompiledGraph {
  // 同步执行，返回最终 State
  invoke(input: State, config?: RunnableConfig): Promise<State>;

  // 流式执行，每个节点完成后 yield 一次
  stream(
    input: State,
    config?: RunnableConfig
  ): AsyncGenerator<Record<string, State>>;

  // 可视化图结构（调试用）
  getGraph(): DrawableGraph;

  // 获取当前状态（需要 Checkpointer）
  getState(config: RunnableConfig): Promise<StateSnapshot>;
}
```

`stream()` 的输出格式：

```
for await (const chunk of graph.stream(input)) {
  // chunk 是一个对象：{ 节点名: 该节点产生的状态更新 }
  // 例如：
  // { llm: { messages: [AIMessage(...)] } }
  // { tools: { messages: [ToolMessage(...)] } }
  console.log(chunk);
}
```

这就是为什么前端能实现「打字机效果」——每个节点完成，前端就能收到一次更新。

总结

这篇我们从底层拆解了 LangGraph 把 LLM 变成状态机的核心机制：

- **StateGraph 三要素**：State 存数据、Node 执行动作、Edge 决定走向，缺一不可
- **Reducer 是关键**：节点返回的是「更新片段」而非「完整新状态」，Reducer 负责合并，State 始终不可变
- **调度器是心脏**：本质是一个事件循环，条件边让它能根据运行时状态动态决策
- **compile() 的意义**：把声明式的图描述变成可执行的调度引擎，做结构验证和邻接表预计算
- **并行靠 Fan-out**：多条出边同时触发，Fan-in 等待所有前驱完成，调度器自动处理同步
- **stream() 是流式基础**：每个节点完成即 yield，这是打字机效果和实时反馈的技术支撑

下一篇我们深入 LangGraph 状态管理的核心：Reducer、Annotation、Channel 是什么关系，以及怎么设计一个合理的 State 结构。

关注我，James 的成长日记，持续分享干货，帮你在 AI 时代少走弯路。

 喜欢作者

AI Agent 成神路 · 目录 ≡

<| 上一篇

RAG 工程实践：分块策略、Rerank、混合检索，这些细节决定效果上限

下一篇 >

LangGraph 状态管理深度解析：Reducer、Annotation、Channel 是什么关系

作者提示: 个人观点，仅供参考